## MIT 8103 Advanced Database Systems: Portfolio Summary

**Case Study:** Digital Banking Platform
**Name:** Osarenkhoe Chima-Nwogwugwu
**Student ID:** 301920803
**GitHub:**grc-cyber-girl/Chima-Nwogwugwu_Osarenkhoe_301920803

## Case study

The five portfolios focused on designing and working with databases for a Digital Banking Platform. The platform supports customer and account management, authentication, fund transfers, beneficiaries and transaction history. The exercises covered relational database design, query processing, transactions, MongoDB and distributed database concepts.

The work used PostgreSQL for structured banking records and MongoDB for transaction event documents. This provided practical experience with different database models and how database design affects data management, retrieval and reliability.

## Portfolio 1: Database Design and Modelling

I designed a relational database for the Digital Banking Platform using PostgreSQL. The database was named `digital_banking_portfolio` and included five main tables: `customer`, `account`, `app_user`, `beneficiary` and `bank_transaction`.

I identified the main users, system operations and business rules before developing the database structure. These rules included unique customer details and account numbers, one account per customer, KYC verification before account activation, and sufficient funds before a transfer could take place.

I used primary keys, foreign keys, unique constraints and check constraints to maintain data integrity. For example, the constraints helped prevent duplicate account numbers and negative account balances. I also recognised that some rules, such as checking a customer's age and ensuring sufficient funds during a transfer, require application or transaction-level logic.

This portfolio helped me understand the importance of planning a database structure before implementation. It also showed how constraints help maintain reliable banking records.

# Portfolio 2: Query Processing and Optimisation

In this portfolio, I wrote SQL queries to retrieve information from the banking database. The queries identified pending transactions, accounts without transactions and the number of transactions associated with each customer.

I also investigated query performance using a benchmark table containing 100,000 synthetic transaction records. I ran the same date-range query before and after creating an index on the `transaction_date` column.

The measured median execution time changed from 6.942 ms to 0.047 ms. The execution plan also changed from a Sequential Scan to an Index Only Scan.

| **Measurement** | **Before index** | **After index** |
|---|---|---|
| Median execution time | 6.942 ms | 0.047 ms |
| Execution plan | Sequential Scan | Index Only Scan |
| Shared buffer hits (third run) | 896 | 3 |

These results showed that the index improved performance for this particular query and test dataset. However, the results do not guarantee the same improvement for every query or database environment. Indexes also require storage and can add overhead when records are inserted or updated.

## Portfolio 3: Transactions and Concurrency

This portfolio focused on how PostgreSQL handles transactions and concurrent database operations.

I used **COMMIT** and **ROLLBACK** to test transaction behaviour. In one exercise, I updated a customer's last login date and committed the change. In another, I attempted an account

balance update that violated the balance constraint. The transaction was rolled back, and the earlier balance reduction within that transaction was undone.

I also used two PostgreSQL sessions at the **READ COMMITTED** isolation level to test concurrent withdrawals. Both sessions attempted to withdraw ₦30,000 from an account with an available balance of ₦50,000. Session A completed its update and committed. Session B waited for the row lock, then returned zero updated rows because the remaining balance was no longer sufficient.

The final balance was ₦20,000. This demonstrated how conditional updates and row locking can help prevent concurrent withdrawals from exceeding an account's available balance. The exercise tested the withdrawal operation, not a complete transfer between two accounts.

## Portfolio 4: NoSQL and Advanced Data Models

I explored MongoDB as a NoSQL database for storing transaction events. I installed MongoDB Community Server in an Ubuntu 24.04 GitHub Codespace and confirmed that the server was running using `db.version()` and a ping command.

I created and queried three transaction event documents in the `digital_banking_events` database. The documents represented different transaction outcomes, including completed, pending and failed transfers. I used MongoDB queries to count the documents, retrieve events associated with a sender and find events with a successful status.

One difficulty I encountered was understanding how the events were structured and how the queries worked. I reviewed the examples and carried out further research to improve my understanding.

This exercise helped me understand how MongoDB stores data in flexible documents and how embedded information can support transaction event records. It also reinforced the importance of choosing a suitable data model for the information being stored.

## Portfolio 5: Distributed and Cloud Database Exercise

This portfolio explored how a distributed database environment could support availability and resilience in a banking platform.

I proposed a MongoDB replica set with three voting members distributed across separate hosts or failure zones. The design included one primary and two secondary members. It explained how elections could take place when a primary becomes unavailable and how a majority of members is required to elect a new primary.

I also considered write concern, particularly `w: "majority"`, and how it affects write acknowledgement. The portfolio also discussed replication, sharding, cloud deployment, security, backup and cost considerations.

The proposed replica set and cloud deployment were design exercises. I did not deploy a three-member replica set or perform failover, replication or cloud performance tests. The work focused on understanding the architecture and its trade-offs.

## Evidence

The GitHub repository contains the portfolio documentation, SQL scripts, MongoDB exercise files and screenshots.

- **Portfolio 1:** Design, ER diagram, normalisation, schema, sample data and constraint test files.
- **Portfolio 2:** SQL query and optimisation scripts, query result screenshots, and before-and-after index execution plan screenshots.
- **Portfolio 3:** Transaction script, Concurrency scripts, and screenshots showing commit, rollback and concurrency results.
- **Portfolio 4:** MongoDB setup documentation, MongoDB scripts, and screenshots showing the MongoDB connection, events and queries.
- **Portfolio 5:** Documentation of the proposed distributed database architecture and its considerations.

All evidence is available in the GitHub repository.

## Individual reflection

Completing these portfolios improved my understanding of database design and implementation. One important design choice was using PostgreSQL for core banking records

and MongoDB for transaction event documents. This helped me appreciate that different database technologies can serve different purposes within the same platform.

I found the events and queries exercise in Portfolio 4 challenging because I initially needed a better understanding of how MongoDB documents were structured and queried. I addressed this by reviewing the examples and conducting further research.

One of my most important lessons was that the efficiency of a good database starts with its design. A well-designed database makes it easier to manage data, retrieve information and maintain consistency.

## References


- PostgreSQL. (2024). *PostgreSQL Documentation: Constraints*. https://www.postgresql.org/docs/16/ddl-constraints.html
- PostgreSQL. *Using EXPLAIN*. https://www.postgresql.org/docs/current/using-explain.html
- PostgreSQL. *Transaction Isolation*. https://www.postgresql.org/docs/current/transaction-iso.html
- PostgreSQL. *Explicit Locking*. https://www.postgresql.org/docs/current/explicit-locking.html
- MongoDB. *Insert Multiple Documents*. https://www.mongodb.com/docs/manual/reference/method/db.collection.insertMany/
- MongoDB. *Query Documents*. https://www.mongodb.com/docs/manual/tutorial/query-documents/
- MongoDB. *Replication*. https://www.mongodb.com/docs/manual/replication/
- MongoDB. *Replica Set Elections*. https://www.mongodb.com/docs/v8.0/core/replica-set-elections/
- MongoDB. *Write Concern*. https://www.mongodb.com/docs/v8.0/reference/write-concern/


## AI-use declaration

I used ChatGPT as a learning and support tool during the completion of the portfolios. It helped explain database concepts, generate fictional data for the exercises and troubleshoot errors encountered during implementation.

I personally ran and reviewed the SQL and MongoDB commands and checked their outputs before using them as evidence. I used the explanations and suggestions to support my understanding and complete the practical exercises. The final work and evidence reflect the commands I executed and the results I reviewed.